

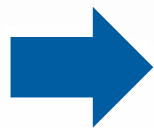
动态规划

讲解人：陆伟

September 15, 2025

目录

CONTENTS



1

动态规划基本思想

2

动态规划适用条件

3

动态规划求解问题基本步骤

4

动态规划求解问题典型案例

5

开放性讨论

6

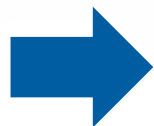
总结

动态规划基本思想

- 动态规划算法与分治法类似，其思想把求解的问题分成许多阶段或多个子问题，然后按顺序求解各子问题。最后一个阶段或子问题的解就是初始问题的解。
- 动态规划中分解得到的子问题往往不是互相独立的。但不同子问题的数目常常只有多项式级。用分治法求解时，有些子问题被重复计算了许多次，从而导致分治法求解问题时间复杂度较高。
- 动态规划基本思想是保留已解决的子问题的解，在需要时再查找已求得的解，就可以避免大量重复计算，进而提升算法效率。

目录

CONTENTS



1

动态规划基本思想

2

动态规划适用条件

3

动态规划求解问题基本步骤

4

动态规划求解问题典型案例

5

开放性讨论

6

总结

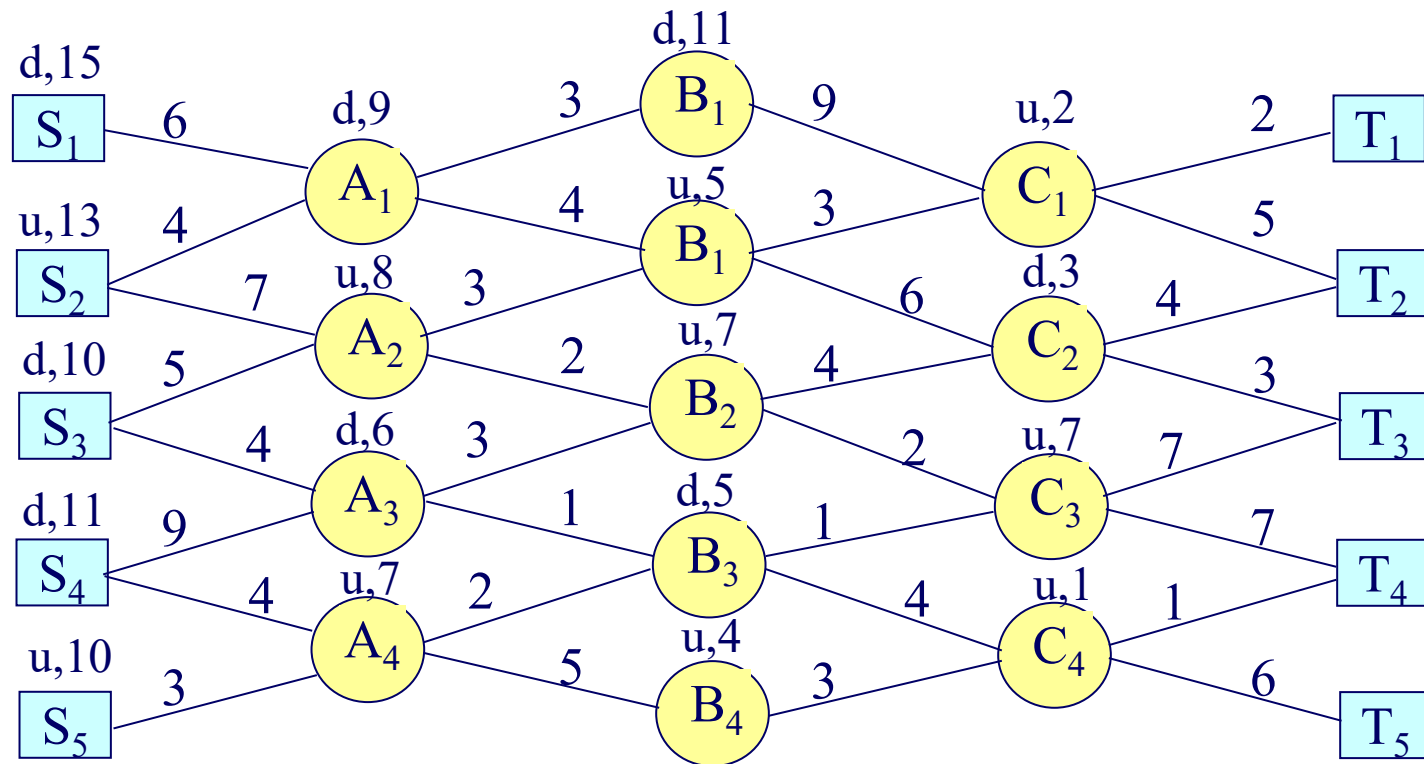


- 

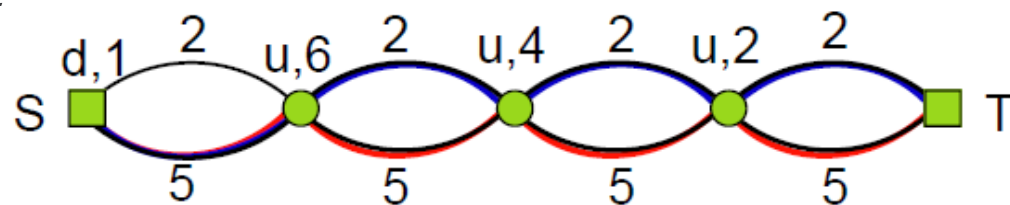
动态规划适用条件

例

□ 多起点、多终点最短路径问题



□ 模10最短路径问题



目录

CONTENTS



1

动态规划基本思想

2

动态规划适用条件

3

动态规划求解问题基本步骤

4

动态规划求解问题典型案例

5

开放性讨论

6

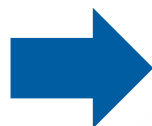
总结

动态规划求解问题计本步骤

- (1) 找出最优解的性质，并刻画其结构特征
- (2) 递推地定义最优值
- (3) 以自底向上的方式计算出最优值
- (4) 根据计算最优值时得到的信息，构造最优解

目录

CONTENTS



1

动态规划基本思想

2

动态规划适用条件

3

动态规划求解问题基本步骤

4

动态规划求解问题典型案例

5

开放性讨论

6

总结

动态规划求解问题典型案例

□ 矩阵连乘问题—问题描述

问题：给定 n 个矩阵 $\{A_1, A_2, \dots, A_n\}$ ，其中， A_i 与 A_{i+1} 是可乘的， $i=1, 2, \dots, n-1$ 。确定矩阵乘法顺序，使得元素相乘的次数最少。

$$\begin{array}{ccccccc} p_0 p_1 & \times & p_1 p_2 & \times & \dots & \times & p_{k-1} p_k & \times & \dots & \times & p_{n-1} p_n \\ A_1 & \times & A_2 & \times & \dots & \times & A_k & \times & \dots & \times & A_n \end{array}$$

实例：3个矩阵 $\{A_1, A_2, A_3\}$ 连乘，设3个矩阵的维数分别为 10×100 , 100×5 , 5×50 。
若按 $((A_1 A_2) A_3)$ 计算，需要数乘次数为：
 $10 \times 100 \times 5 + 10 \times 5 \times 50 = 7500$ ；
若按 $(A_1 (A_2 A_3))$ 计算，需要数乘次数为：
 $100 \times 5 \times 50 + 10 \times 100 \times 50 = 75000$ 。
矩阵连乘次序对计算量有很大影响。

动态规划求解问题典型案例

□ 矩阵连乘问题 – 初步尝试

蛮力法：搜索所有可能的计算次序，并计算出每种计算次序相应需要的数乘次数，从中找出一种数乘次数最少的计算次序。设不同计算次序为 $P(n)$ 。

复杂度分析

$$P(n) = \begin{cases} 1 & n = 1 \\ \sum_{k=1}^{n-1} P(k)P(n-k) & n > 1 \end{cases}$$

该递推方程解为Catalan数 ($C(n) = \frac{1}{n+1} \binom{2n}{n}$)
 $P(n) = \Omega(4^n/n^{3/2})$

动态规划求解问题典型案例

□ 矩阵连乘问题—深入分析

将矩阵连乘积 $A_i A_{i+1} \dots A_j$ 简记为 $A[i:j]$, $i \leq j$ 。

考察计算 $A[i:j]$ 的最优计算次序。设这个计算次序在矩阵 A_k 和 A_{k+1} 之间将矩阵链断开, $i \leq k < j$, 则其相应完全加括号方式为 $(A_i A_{i+1} \dots A_k)(A_{k+1} A_{k+2} \dots A_j)$

计算量:

$A[i:k]$ 的计算量加上 $A[k+1:j]$ 的计算量, 再加上 $A[i:k]$ 和 $A[k+1:j]$ 相乘的计算量。

(1) 最优解结构性质的分析:

- 计算 $A[i:j]$ 的最优次序所包含的计算矩阵子链 $A[i:k]$ 和 $A[k+1:j]$ 的次序也是最优的。
- 矩阵连乘计算次序问题的最优解包含着其子问题的最优解, 满足最优子结构性质。问题的最优子结构性质是该问题可用动态规划算法求解的显著特征。

动态规划求解问题典型案例

□ 矩阵连乘问题－动态规划

(2) 建立递推关系

1. 设计算 $A[i:j]$, $1 \leq i \leq j \leq n$, 所需要的最少数乘次数 $m[i, j]$, 则原问题的最优值为 $m[1, n]$ 。

2. 递推方程

$$m[i, j] = \begin{cases} 0 & i = j \\ \min_{i \leq k < j} \{m[i, k] + m[k+1, j] + p_{i-1}p_kp_j\} & i < j \end{cases}$$

推导

K的位置只有j-i种可能

$$\begin{array}{ccccccc} p_{i-1}p_i & \times & p_ip_{i+1} & \times & \dots & \times & p_{k-1}p_k & \times & \dots & \times & p_{j-1}p_j \\ A_i & \times & A_{i+1} & \times & \dots & \times & A_k & \times & \dots & \times & A_j \end{array}$$

动态规划求解问题典型案例

□ 矩阵连乘问题—递归求解?

$S[i][j]$ 表示矩阵链 $A[i:j]$
最少乘次的断开位置。

(3) 计算最优值—递归求解?

```
int RecurMatrixChain(int i, int j, int *p, int **s) {  
    if(i == j) return 0;  
    int u = RecurMatrixChain(i, i) + RecurMatrixChain(i+1, j) + p[i-1]*p[i]*p[j];  
    s[i][j] = i;  
    for(int k = i+1; k < j; k++){  
        int t = RecurMatrixChain(i, k) + RecurMatrixChain(k+1, j) + p[i-1]*p[k]*p[j];  
        if(t < u) {u = t; s[i][j] = k;}  
    }  
    return u;  
}
```

实践

$$T(n) \geq \begin{cases} O(1) & n = 1 \\ \sum_{k=1}^{n-1} (T(k) + T(n-k) + O(1)) & n > 1 \end{cases}$$

$$T(n) \geq O(n) + \sum_{k=1}^{n-1} T(k) + \sum_{k=1}^{n-1} T(n-k) = O(n) + 2 \sum_{k=1}^{n-1} T(k)$$

$$T(n) \geq 2^{n-1}$$

动态规划求解问题典型案例

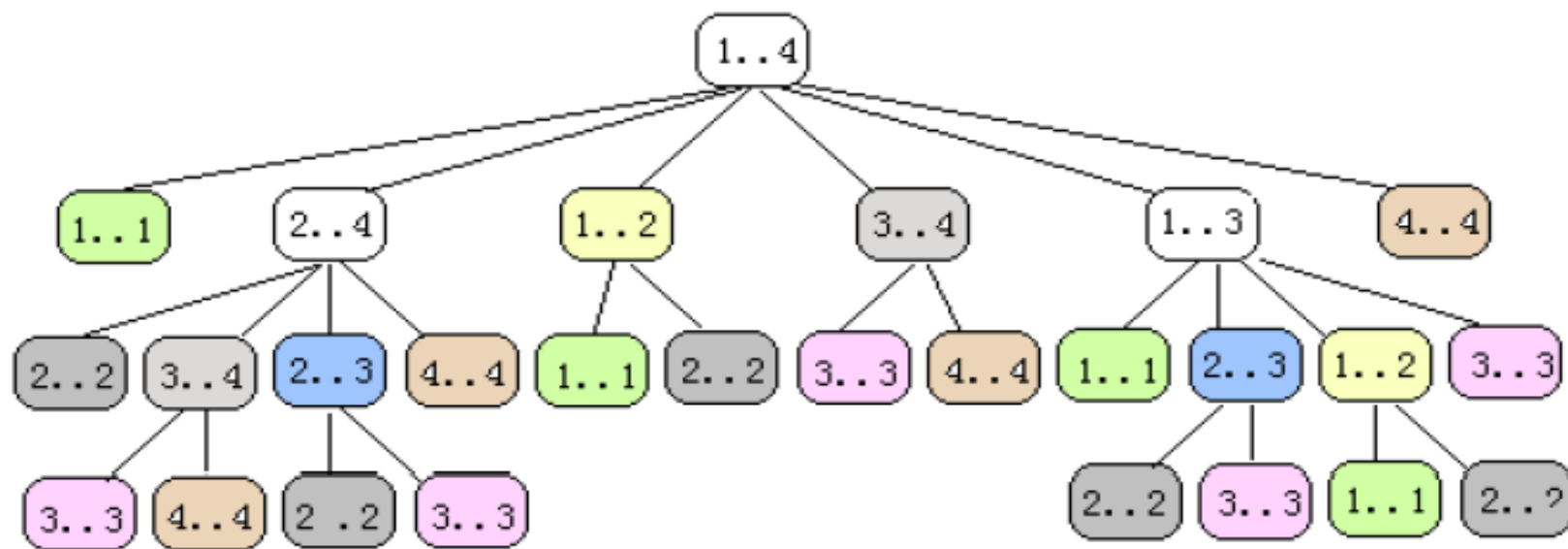
□ 矩阵连乘问题 – 递归求解分析

对于 $1 \leq i \leq j \leq n$ 不同的有序对 (i, j) 对应于不同的子问题, 不同子问题的个数最多只有:

$$\binom{n}{2} + n = \Theta(n^2)$$

递归求解最优值复杂度较高的原因是: 子问题重复度高

讨论与
理解



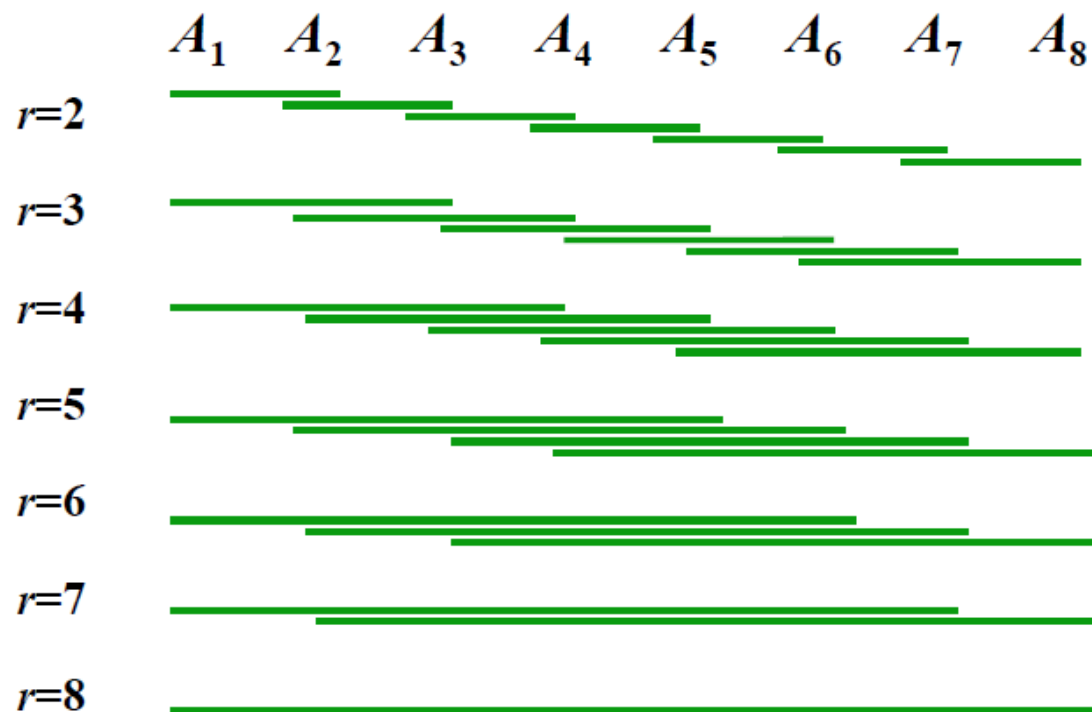
动态规划求解问题典型案例

□ 矩阵连乘问题—自底向上求解最优值

(3) 计算最优值—迭代查表求解

```
void MatrixChain(int *p, int n, int **m, int **s) {  
    for (int i = 1; i <= n; i++) m[i][i] = 0;  
    for (int r = 2; r <= n; r++)  
        for (int i = 1; i <= n-r+1; i++) {  
            int j = i+r-1;  
            m[i][j] = m[i+1][j] + p[i-1]*p[i]*p[j];  
            s[i][j] = i;  
            for (int k = i+1; k < j; k++) {  
                int t = m[i][k] + m[k+1][j] + p[i-1]*p[k]*p[j];  
                if (t < m[i][j]) {  
                    m[i][j] = t;  
                    s[i][j] = k;  
                }  
            }  
        }  
}
```

实践



时间复杂度 $O(n^3)$
空间复杂的 $O(n^2)$

参考代码

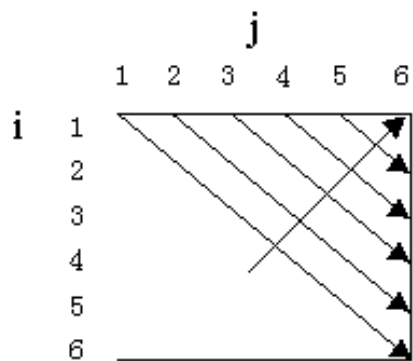
动态规划求解问题典型案例

例

□ 矩阵连乘问题－算例分析

A1	A2	A3	A4	A5	A6
30×35	35×15	15×5	5×10	10×20	20×25

$$m[2][5] = \min \begin{cases} m[2][2] + m[3][5] + p_1 p_2 p_5 = 0 + 2500 + 35 \times 15 \times 20 = 13000 \\ m[2][3] + m[4][5] + p_1 p_3 p_5 = 2625 + 1000 + 35 \times 5 \times 20 = 7125 \\ m[2][4] + m[5][5] + p_1 p_4 p_5 = 4375 + 0 + 35 \times 10 \times 20 = 11375 \end{cases}$$



(a) 计算次序

	j	1	2	3	4	5	6
i	1	0	15750	7875	9375	11875	15125
	2		0	2625	4375	7125	10500
	3			0	750	2500	5375
	4				0	1000	3500
	5					0	5000
	6						0

(b) $m[i][j]$

	j	1	2	3	4	5	6
i	1	0	1	1	3	3	3
	2		0	2	3	3	3
	3			0	3	3	3
	4				0	4	5
	5					0	5
	6						0

(c) $s[i][j]$

动态规划求解问题典型案例

□ 矩阵连乘问题—构造最优解

(4) 构造最优解

```
void Tracback (int i, int j, int **s) {  
    if (i == j) return;  
    Tracback (i, s[i][j], s);  
    Tracback (s[i][j] + 1, j, s);  
    cout << "Multiply" << i << "," << s[i][j];  
    cout << "and A" << (s[i][j]) + 1 << "," << j << endl;  
}
```

设输入 $P=\langle 30, 35, 15, 5, 10, 20, 25 \rangle$, $n=6$, 相应矩阵链为:
 $A_1, A_2, A_3, A_4, A_5, A_6$ 其中,
 $A_1:30\times35, A_2:35\times15, A_3:15\times5, A_4:5\times10, A_5:10\times20, A_6:20\times25$

例

动态规划求解问题典型案例

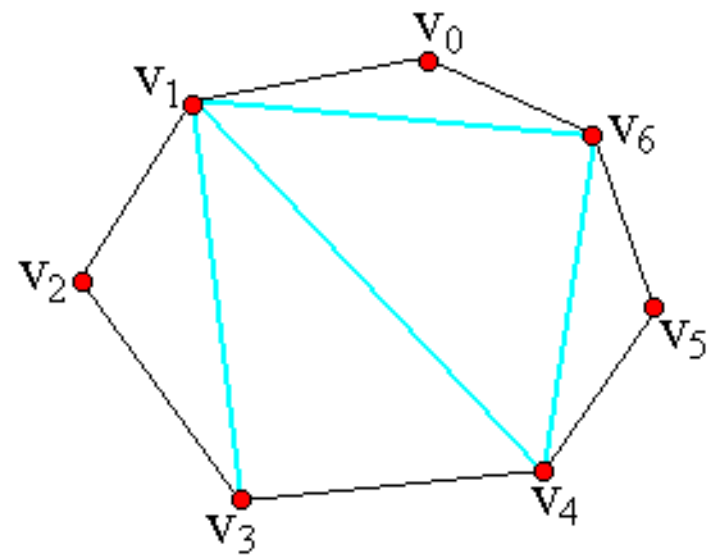
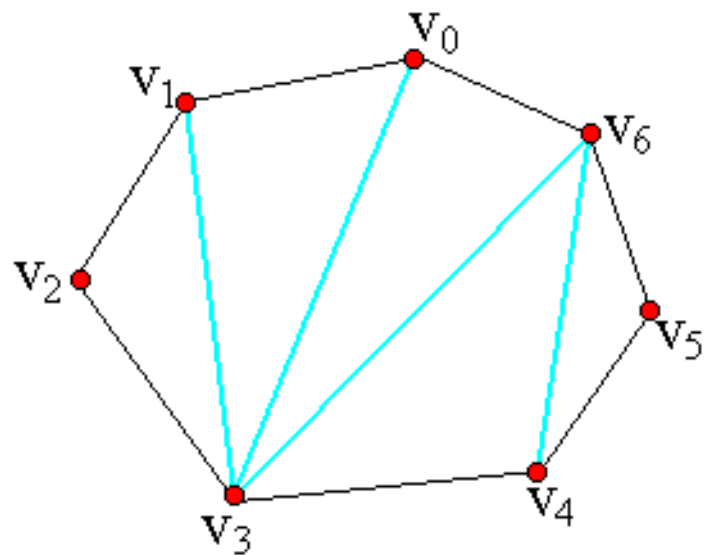
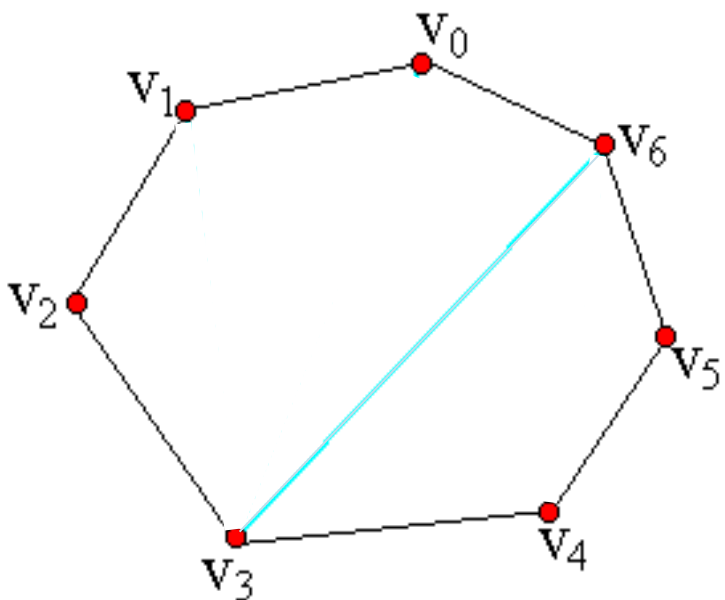
□ 凸多边形最优三角剖分

凸多边形的表示：多边形顶点的逆时针序列表示。

例： $P=\{v_0, v_1, v_2, v_3, v_4, v_5, v_6\}$

弦：两个不相邻顶点间的直线线段。弦将多边形分割为两个子多边形。

多边形的三角剖分指将多边形分割成互不相交的三角形的弦的集合 T 。 n 个顶点的凸多边形剖分必有 $n-3$ 条弦和 $n-2$ 个三角形。



动态规划求解问题典型案例

□ 凸多边形最优三角剖分

问题：给定凸多边形 $P=\{v_0, v_1, \dots, v_{n-1}\}$ ，以及定义在由凸多边形的边和弦组成的三角形上的权函数 w 。要求确定该凸多边形的三角剖分，使得该三角剖分所对应的权，即三角剖分中诸三角形上权之和为最小。

(1) 一个表达式的完全加括号方式相应于一棵完全二叉树，称为表达式的语法树。例如，完全加括号的矩阵链乘积 $((A_1(A_2A_3))(A_4(A_5A_6)))$ 所相应的语法树如图 (a) 所示。

(2) 凸多边形 $\{v_0, v_1, \dots, v_{n-1}\}$ 的三角剖分也可以用语法树表示。例如，图 (b) 中凸多边形的三角剖分可用图 (a) 所示的语法树表示。

(3) 矩阵链乘积中的每个矩阵 A_i 对应于凸 $(n+1)$ 边形中的一条边 $v_{i-1}v_i$ 。三角剖分中的一条弦 v_iv_j ， $i < j$ ，对应于矩阵链乘积 $A[i+1:j]$ 。

(4) 矩阵链乘的最优计算次序问题是凸多边形最优三角剖分问题的特殊情形。

动态规划求解问题典型案例

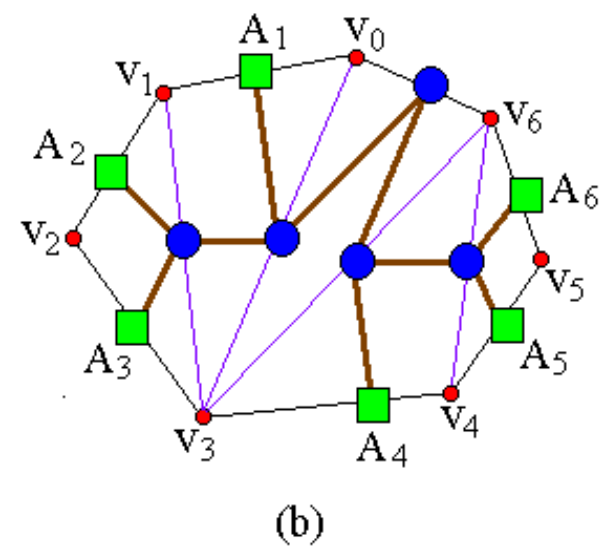
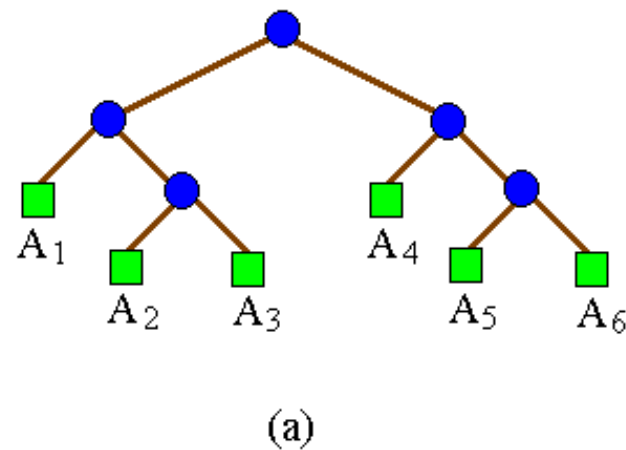
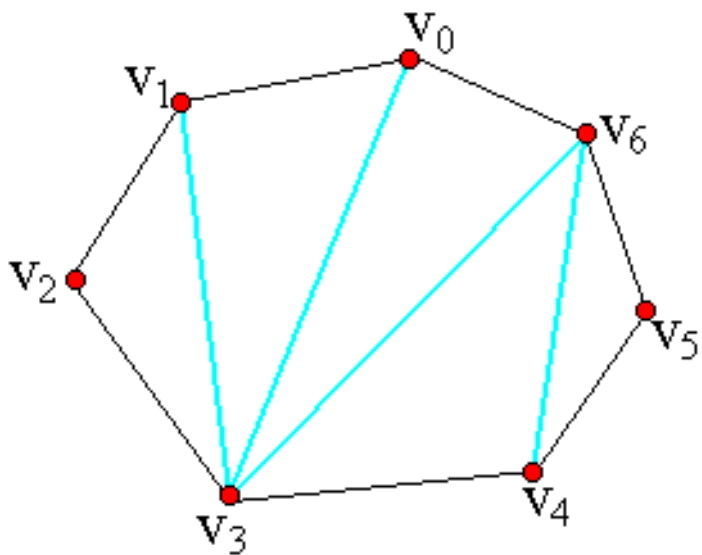
□ 凸多边形最优三角剖分 – 最优子结构性质分析

凸多边形的最优三角剖分问题有最优子结构性质。

若凸 $(n+1)$ 边形 $P=\{v_0, v_1, \dots, v_n\}$ 的最优三角剖分 T 包含三角形 $v_0 v_k v_n$, $1 \leq k \leq n-1$, 则 T 的权为3个部分权的和: 三角形 $v_0 v_k v_n$ 的权, 子多边形 $\{v_0, v_1, \dots, v_k\}$ 和 $\{v_k, v_{k+1}, \dots, v_n\}$ 的权之和。

由 T 所确定的这2个子多边形的三角剖分也是最优的。

因为若有 $\{v_0, v_1, \dots, v_k\}$ 或 $\{v_k, v_{k+1}, \dots, v_n\}$ 的更小权的三角剖分将导致 T 不是最优三角剖分的矛盾。



动态规划求解问题典型案例

□ 凸多边形最优三角剖分 – 同构问题

定义 $t[i][j]$, $1 \leq i < j \leq n$ 为凸子多边形 $\{v_{i-1}, v_i, \dots, v_j\}$ 的最优三角剖分所对应的权函数值, 即其最优值。为方便起见, 设退化的多边形 $\{v_{i-1}, v_i\}$ 具有权值0。据此定义, 要计算的凸 $(n+1)$ 边形 P 的最优权值为 $t[1][n]$ 。

$t[i][j]$ 的值可以利用最优子结构性质递归地计算。当 $j-i \geq 1$ 时, 凸子多边形至少有3个顶点。由最优子结构性质, $t[i][j]$ 的值应为 $t[i][k]$ 的值加上 $t[k+1][j]$ 的值, 再加上三角形 $v_{i-1}v_kv_j$ 的权值, 其中 $i \leq k \leq j-1$ 。由于在计算时还不知道 k 的确切位置, 而 k 的所有可能位置只有 $j-i$ 个, 因此可以在这 $j-i$ 个位置中选出使 $t[i][j]$ 值达到最小的位置。

$$t[i][j] = \begin{cases} 0 & i = j \\ \min_{i \leq k < j} \{t[i][k] + t[k+1][j] + w(v_{i-1}v_kv_j)\} & i < j \end{cases}$$

凸多边形最优三角剖分

$$m[i, j] = \begin{cases} 0 & i = j \\ \min_{i \leq k < j} \{m[i, k] + m[k+1, j] + p_{i-1}p_kp_j\} & i < j \end{cases}$$

矩阵链最优乘次

动态规划求解问题典型案例

□ 最长公共子序列问题

若给定序列 $X=\{x_1, x_2, \dots, x_m\}$ ，则另一序列 $Z=\{z_1, z_2, \dots, z_k\}$ 是 X 的子序列是指，存在一个严格递增下标序列 $\{i_1, i_2, \dots, i_k\}$ 使得对于所有 $j=1, 2, \dots, k$ 有： $z_j=x_{i_j}$ 。

给定2个序列 X 和 Y ，当另一序列 Z 既是 X 的子序列又是 Y 的子序列时，称 Z 是序列 X 和 Y 的**公共子序列**。

$Z=\{B, C, D, B\}$ 是序列 $X=\{A, B, C, B, D, A, B\}$ 的子序列，相应的递增下标序列为 $\{2, 3, 5, 7\}$

$X=\{A, B, C, B, D, A, B\}$ ， $Y=\{B, D, C, A, B, A\}$ ，则序列 $Z=\{B, C, A\}$ 是 X 和 Y 的一个公共子序列。

序列 $W=\{B, C, B, A\}$ 是 X 和 Y 的一个公共子序列，并且是 X 和 Y 的最长公共子序列。

问题：给定2个序列 $X=\{x_1, x_2, \dots, x_m\}$ 和 $Y=\{y_1, y_2, \dots, y_n\}$ ，找出 X 和 Y 的最长公共子序列。

蛮力法：求 X 和 Y 的所有公共子序列，找出最长的。

动态规划求解问题典型案例

□ 最长公共子序列问题 – 最优子结构性质分析

(1) 最优子结构性质

设序列 $X=\{x_1, x_2, \dots, x_m\}$ 和 $Y=\{y_1, y_2, \dots, y_n\}$ 的最长公共子序列为 $Z=\{z_1, z_2, \dots, z_k\}$ ，则

(a) 若 $x_m = y_n$ ，则 $z_k = x_m = y_n$ ，且 z_{k-1} 是 x_{m-1} 和 y_{n-1} 的最长公共子序列。

(b) 若 $x_m \neq y_n$ 且 $z_k \neq x_m$ ，则 Z 是 x_{m-1} 和 Y 的最长公共子序列。

(c) 若 $x_m \neq y_n$ 且 $z_k \neq y_n$ ，则 Z 是 X 和 y_{n-1} 的最长公共子序列。

由此可见，2个序列的最长公共子序列包含了这2个序列的前缀的最长公共子序列。
因此，最长公共子序列问题具有**最优子结构性质**。

动态规划求解问题典型案例

□ 最长公共子序列问题 – 递推方程建立

(2) 建立最优解递推关系

用 $c[i][j]$ 记录序列 X_i 和 Y_j 的最长公共子序列的长度。其中, $X_i=\{x_1,x_2,\dots,x_i\}$; $Y_j=\{y_1,y_2,\dots,y_j\}$ 。当 $i=0$ 或 $j=0$ 时, 空序列是 X_i 和 Y_j 的最长公共子序列。故此时 $C[i][j]=0$ 。其它情况下, 由最优子结构性性质可建立递归关系如下:

$$c[i][j] = \begin{cases} 0 & i = 0, j = 0 \\ c[i-1][j-1] + 1 & i, j > 0; x_i = y_j \\ \max \{c[i][j-1], c[i-1][j]\} & i, j > 0; x_i \neq y_j \end{cases}$$

推导

动态规划求解问题典型案例

□ 最长公共子序列问题 – 自底向上求解最优值

(3) 计算最优值

子问题空间总共有 $\theta(mn)$ 个不同的子问题，因此，用动态规划算法自底向上地计算最优值能提高算法的效率。

```
void LCSLength(int m, int n, char *x, char *y, int **c, int **b) {
    int i, j;
    for (i = 1; i <= m; i++) c[i][0] = 0;
    for (i = 1; i <= n; i++) c[0][i] = 0;
    for (i = 1; i <= m; i++)
        for (j = 1; j <= n; j++) {
            if (x[i]==y[j]) {
                c[i][j]=c[i-1][j-1]+1; b[i][j]=1;}
            else if (c[i-1][j]>=c[i][j-1]) {
                c[i][j]=c[i-1][j]; b[i][j]=2;}
            else { c[i][j]=c[i][j-1]; b[i][j]=3; }
        }
}
```

$C[i][j]$ 存储 X_i 和 Y_j 的最长公共子序列的长度， $b[i][j]$ 记录 $C[i][j]$ 的值是由哪一个子问题的解得到的，后面构造最长公共子序列时需要用到。

算法时间复杂度为 $O(mn)$

参考代码

动态规划求解问题典型案例

□ 最长公共子序列问题—构造最优解

(4) 构造最优解

从 $b[m][n]$ 开始，在数组 b 中搜索，当 $b[i][j]=1$ 时，表示 X_i 和 Y_j 的最长公共子序列是由 X_{i-1} 和 Y_{j-1} 的最长公共子序列在尾部加上 x_i 所得到的子序列；当 $b[i][j]=2$ 时，表示表示 X_i 和 Y_j 的最长公共子序列与 X_{i-1} 和 Y_j 的最长子序列相同；当 $b[i][j]=3$ 时，表示表示 X_i 和 Y_j 的最长公共子序列与 X_i 和 Y_{j-1} 的最长子序列相同。

```
void LCS(int i, int j, char *x, int **b) {  
    if (i == 0 || j == 0) return;  
    if (b[i][j] == 1) {  
        LCS(i-1, j-1, x, b);  
        cout << x[i];  
    }  
    else if (b[i][j] == 2) LCS(i-1, j, x, b);  
    else LCS(i, j-1, x, b);  
}
```

算法时间复杂度为 $O(m+n)$

动态规划求解问题典型案例

例

□ 最长公共子序列问题－算例

		B	D	C	A	B	A
	c[m][n]	0	0	0	0	0	0
A	0	0	0	0	1	1	1
B	0	1	1	1	1	2	2
C	0	1	1	2	2	2	2
B	0	1	1	2	2	3	3
D	0	1	2	2	2	3	3
A	0	1	2	2	3	3	4
B	0	1	2	2	3	4	4

动态规划求解问题典型案例

例

□ 最长公共子序列问题 – 算例分析

$X=\{A, B, C, B, D, A, B\}$, $Y=\{B, D, C, A, B, A\}$

b[m][n]	1B	2D	3C	4A	5B	6A
1A	2	2	2	1	3	1
2B	1	3	3	2	1	3
3C	2	2	1	3	2	2
4B	2	2	2	2	1	3
5D	2	2	2	2	2	2
6A	2	2	2	1	2	1
7B	2	2	2	2	2	2

1 ↖
2 ↑
3 ←

动态规划求解问题典型案例

□ 最长公共子序列问题 – 同构问题

问题：最长单调递增子序列。

设 $L=\{a_1, a_2, \dots, a_n\}$ 是 n 个不同的实数序列， L 的递增子序列是指 $L'=\{a_{k1}, a_{k2}, \dots, a_{km}\}$ ，其中 $k1 < k2 < \dots < km$ ，且 $a_{k1} \leq a_{k2} \leq \dots \leq a_{km}$ ，求最大的 m 值。

25	118	130	115	199	260	167	280

$$d[i] = \max_{\substack{0 \leq k < i \\ s[i] \geq s[k]}} d[k] + 1 \quad d[0:n-1] = 1$$

动态规划求解问题典型案例

□ 最长公共子序列问题—同构问题

问题：序列匹配。

设 $S_1[1,m]$ 和 $S_2[1,n]$ 为两个字符序列，对这两个字符序列进行顺序扫描比较，比较时可以在某个序列的两个字符之间插入空格，以使它们局部相同区域对应。在比较中，如果两个对应字符相等，赋权值2，如果对应字符不等，赋权值-2，如果一个是一个字符一个是空格，赋权值-1。

问题：寻找权值最大的匹配度（匹配权值和）与对准方式。

例：序列axabcdes和axbacfes的两种对准方式如下：

a x a b - c d e s
a x - b a c f e s

a x - a b c d e s
a x b a - c f e s

这两种对准方式的匹配度均为： $6 \times 2 + (-2) + 2 \times (-1) = 8$

动态规划求解问题典型案例

□ 最长公共子序列问题 – 同构问题

设 $c[i,j]$ 表示序列 $s_1[1,i]$ 与 $s_2[1,j]$ 对比的最大权值。

(1) $s_1[i]$ 与 $s_2[j]$ 对准情况:

a) 若 $s_1[i] = s_2[j]$, 问题规约为 $s_1[1,i-1]$ 与 $s_2[1,j-1]$ 的对准, 权值为 $c[i-1,j-1]+2$;

b) 若 $s_1[i] \neq s_2[j]$, 问题仍规约为 $s_1[1,i-1]$ 与 $s_2[1,j-1]$ 的对准, 权值为 $c[i-1,j-1]-2$;

(2) $s_1[i]$ 与 $s_2[j]$ 不对准情况:

需要在序列 $s_1[1,i]$ 或 $s_2[1,j]$ 后增加空格, 问题归结为 $s_1[1,i-1]$ 与 $s_2[1,j]$ 的对准, 或者 $s_1[1,i]$ 与 $s_2[1,j-1]$ 的对准, 权值为子问题权值-1.

$$c[i][j] = \max_{\substack{0 \leq i \leq m \\ 0 \leq j \leq n}} \{0, c[i-1][j-1] + t(s_1[i], s_2[j]), c[i][j-1] - 1, c[i-1][j] - 1\}$$

时间复杂度 $O(mn)$

$$t(s_1[i], s_2[j]) = \begin{cases} 2 & s_1[i] = s_2[j] \\ -2 & s_1[i] \neq s_2[j] \end{cases} \quad \begin{matrix} c[0][j] = 0 \\ c[i][0] = 0 \end{matrix}$$

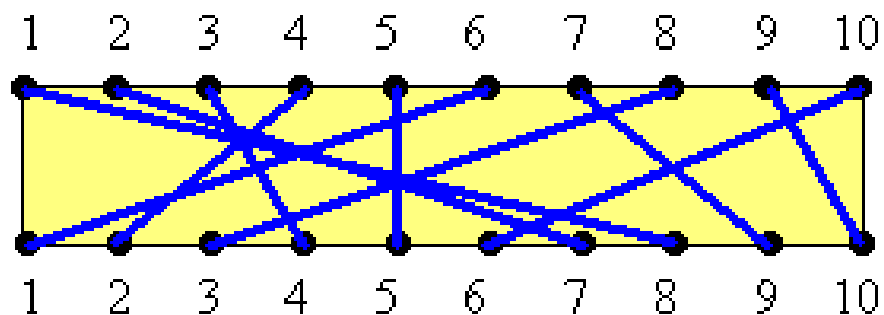
课堂演示

动态规划求解问题典型案例

□ 电路布线问题

在一块电路板的上、下两端分别有 n 个接线柱。根据电路设计，要求用导线 $(i, \pi(i))$ 将上端接线柱与下端接线柱相连，其中 $\pi(i)$ 是 $\{1, 2, \dots, n\}$ 的一个排列。导线 $(i, \pi(i))$ 称为该电路板上的第 i 条连线。对于任何 $1 \leq i < j \leq n$ ，第 i 条连线和第 j 条连线相交的充分且必要的条件是 $\pi(i) > \pi(j)$ 。

电路布线问题要确定将哪些连线安排在第一层上，使得该层上有尽可能多的连线。换句话说，该问题要求确定导线集 $\text{Nets} = \{(i, \pi(i)), 1 \leq i \leq n\}$ 的最大不相交子集。



动态规划求解问题典型案例

□ 电路布线问题－最优子结构性质的分析

记 $N(i, j) = \{t \mid (t, \pi(t)) \in Nets, t \leq i, \pi(t) \leq j\}$, $N(i, j)$ 的最大不相交子集为 $MNS(i, j)$ 。
 $Size(i, j) = |MNS(i, j)|$ 。

(1) 当 $i=1$ 时

$$Size(1, j) = \begin{cases} 0 & j < \pi(1) \\ 1 & j \geq \pi(1) \end{cases}$$

(2) 当 $i > 1$ 时

$$Size(i, j) = \begin{cases} Size(i-1, j) & j < \pi(i) \\ \max \{Size(i-1, j), Size(i-1, \pi(i)-1) + 1\} & j \geq \pi(i) \end{cases}$$

与最长公共子序列问题同构

动态规划求解问题典型案例

□ 电路布线问题

```
void MNS (int C[], int n, int** size) {  
    for (int j = 0; j < C[1]; j++) size[1][j] = 0;  
    for (int j = C[1]; j <= n; j++) size[1][j] = 1;  
    for (int i = 2; i < n; i++) {  
        for (int j = 0; j < C[i]; j++) size[i][j] = size[i-1][j];  
        for (int j = C[i]; j <= n; j++)  
            size[i][j] = max(size[i-1][j], size[i-1][C[i]-1]+1);  
    }  
    size[n][n] = max(size[n-1][n], size[n-1][C[n]-1]+1);  
}
```

时间复杂度 $O(n^2)$

```
void Traceback (int C[], int** size, int n, int Net[], int& m) {  
    int j = n;  
    m = 0;  
    for (int i = n; i > 1; i--)  
        if (size[i][j] != size[i-1][j]) {  
            Net[m++] = i; j = C[i] - 1;  
        }  
    if (j >= C[1]) Net[m++] = 1;  
}
```

时间复杂度 $O(n)$

动态规划求解问题典型案例

□ 图像压缩问题

图象的变位压缩存储格式将所给的象素点序列 $\{p_1, p_2, \dots, p_n\}$, $0 \leq p_i \leq 255$ 分割成 m 个连续段 $S_1, S_2, \dots, S_m$ 。第 i 个象素段 S_i 中 ($1 \leq i \leq m$), 有 $l[i]$ 个象素, 且该段中每个象素都只用 $b[i]$ 位表示。设 $t[i] = \sum_{k=1}^{i-1} l[k]$, 则第 i 个象素段 S_i 为 $S_i = \{p_{t[i]+1}, \dots, p_{t[i]+l[i]}\}$ 。

$$\text{设 } h_i = \left\lceil \log \left(\max_{t[i]+1 \leq k \leq t[i]+l[i]} p_k + 1 \right) \right\rceil$$

则 $h_i \leq b[i] \leq 8$ 。因此需要用 3 位表示 $b[i]$, 如果限制 $1 \leq l[i] \leq 255$, 则需要用 8 位表示 $l[i]$ 。因此, 第 i 个象素段所需的存储空间为 $l[i] * b[i] + 11$ 位。按此格式存储象素序列 $\{p_1, p_2, \dots, p_n\}$, 需要位存储空间为 $\sum_{i=1}^m l[i] * b[i] + 11m$

动态规划求解问题典型案例

□ 图像压缩问题

图象压缩问题要求确定象素序列 $\{p_1, p_2, \dots, p_n\}$ 的最优分段，使得依此分段所需的存储空间最少。每个分段的长度不超过256位。

设输入的灰度值为 $P = \{10, 12, 15, 255, 1, 2, 1, 1, 2, 2, 1, 1\}$,

划分1: $S_1 = \{10, 12, 15\}, S_2 = \{255\}, S_3 = \{1, 2, 1, 1, 2, 2, 1, 1\}$

划分2: $S_1 = \{10, 12, 15, 255, 1, 2, 1, 1, 2, 2, 1, 1\}$

划分3: $S_1 = \{10\}, S_2 = \{12\}, S_3 = \{15\}, S_4 = \{255\}, S_5 = \{1\}, S_6 = \{2\},$
 $S_7 = \{1\}, S_8 = \{1\}, S_9 = \{2\}, S_{10} = \{2\}, S_{11} = \{1\}, S_{12} = \{1\}$

不同划分占用位数:

段头11位

划分1: $4 \times 3 + 8 \times 1 + 2 \times 8 + 11 \times 3 = 69$

划分2: $8 \times 12 + 11 \times 1 = 107$

划分3: $4 + 4 + 4 + 8 + 1 + 1 + 1 + 1 + 1 + 1 + 1 + 1 + 11 \times 12 = 163$

不同划分压缩效果不同，如何寻找最优划分结果

动态规划求解问题典型案例

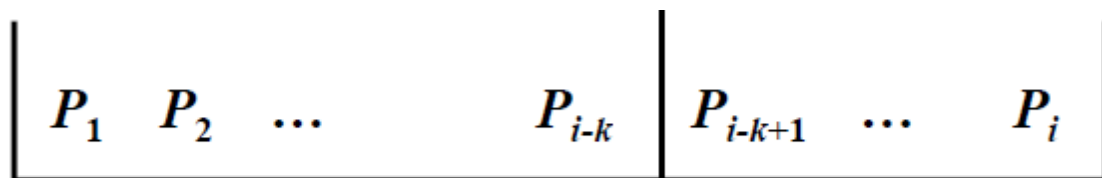
□ 图像压缩问题－最优子结构性质分析－递推定义最优值

设 $l[i], b[i]$, $1 \leq i \leq m$ 是 $\{p_1, p_2, \dots, p_n\}$ 的一个最优分段。 $l[1], b[1]$ 则是 $\{p_1, \dots, p_{l[1]}\}$ 的一个最优分段, 且 $l[i], b[i]$, $2 \leq i \leq m$, 是 $\{p_{l[1]+1}, \dots, p_n\}$ 的一个最优分段。即
图像压缩问题满足最优子结构性质。

设 $s[i]$, $1 \leq i \leq n$, 是象素序列 $\{p_1, \dots, p_i\}$ 的最优分段所需的存储位数。

$$s[i] = \min_{1 \leq k \leq \min\{i, 256\}} \{s[i-k] + k * b_{\max(i-k+1, i)}\} + 1$$

$$b_{\max(i, j)} = \left\lceil \log \left(\max_{i \leq k \leq j} \{p_k\} + 1 \right) \right\rceil$$



$S[i-k]$ 位

k 个灰度
 $k * b_{\max(i-k+1, i)}$

动态规划求解问题典型案例

□ 图像压缩问题 – 自底向上求解最优值，构造最优解

```
void Compress (int n, int p[], int s[], int l[], int b[]){
    int Lmax = 256, header = 11;
    s[0] = 0;
    for (int i; i<=n; i++) {
        b[i] = length(p[i]);
        int bmax = b[i];
        s[i] = s[i-1] + bmax;
        l[i] = 1;
        for (int j = 2; j <= i && j <= Lmax; j++){
            if (bmax < b[i-j+1]) bmax = b[i-j+1];
            if (s[i] > s[i-j] + j*bmax) {
                s[i] = s[i-j] + j*bmax;
                l[i] = j;
            }
        }
        s[i] += header;
    }
}
```

```
int length (int i){
    int k = 1; i = i/2;
    while (i > 0) {
        k++;
        i = i/2;
    }
    return k;
}
```

```
void Traceback (int n, int& L, int s[], int l[]) {
    if (n == 0) return;
    Traceback (n - l[n], l, s, l);
    s[l++] = n - l[n];
}
```

```
void Output (int s[], int l[], int b[], int n) {
    cout << "The optimal value is " << s[n] << endl;
    int m = 0;
    Traceback (n, m, s, l);
    s[m] = n;
    cout << "Decompose into " << m << "segments" << endl;
    for (int j = 1; j <= m; j++) {
        l[j] = l[s[j]]; b[j] = b[s[j]];
    }
    for (int j = 1; j<=m; j++) {
        cout << l[j] << ' ' << b[j] << endl;
    }
}
```

由于算法**Compress**中对k的循环次数不超这256，故对每一个确定的i，可在时间 $O(1)$ 内完成的计算。因此整个算法所需的计算时间为 $O(n)$ 。

参考代码

动态规划求解问题典型案例

例

□ 图像压缩问题－算例分析

$P = \{10, 12, 15, 255, 1, 2\}$

$\{10, 12, 15\}, \{255\}, \{1, 2\}$

$s[0] = 0$
 $s[1] = s[0] + 1 * bmax(p[1]) + header = 0 + 4 + 11 = 15; \quad l[1] = 1;$
 $s[2] = \min\{s[1] + 1 * bmax(p[2]), s[0] + 2 * bmax(p[1], p[2])\} + header = 19; \quad l[2] = 2;$
 $s[3] = \min\{s[2] + 1 * bmax(p[3]), s[1] + 2 * bmax(p[2], p[3]), s[0] + 3 * bmax(p[1], p[2], p[3])\} + header = 23; \quad l[3] = 3;$
 $s[4] = \min\{s[3] + 1 * bmax(p[4]), s[2] + 2 * bmax(p[3], p[4]), s[1] + 3 * bmax(p[2], p[3], p[4]),$
 $\quad s[0] + 4 * bmax(p[1], p[2], p[3], p[4])\} + header = 42; \quad l[4] = 1;$
 $s[5] = \min\{s[4] + 1 * bmax(p[5]), s[3] + 2 * bmax(p[4], p[5]), s[2] + 3 * bmax(p[3], p[4], p[5]),$
 $\quad s[1] + 4 * bmax(p[2], p[3], p[4], p[5]), s[0] + 5 * bmax(p[1], p[2], p[3], p[4], p[5])\} + header = 50; \quad l[5] = 2;$
 $s[6] = \min\{s[5] + 1 * bmax(p[6]), s[4] + 2 * bmax(p[5], p[6]), s[3] + 3 * bmax(p[4], p[5], p[6]),$
 $\quad s[2] + 4 * bmax(p[3], p[4], p[5], p[6]), s[1] + 5 * bmax(p[2], p[3], p[4], p[5], p[6]), s[0] +$
 $\quad 6 * bmax(p[1], p[2], p[3], p[4], p[5], p[6]), \} + header = 57; \quad l[6] = 2;$

动态规划求解问题典型案例

□ 最大字段和问题

问题：给定 n 个整数（可以为负数）的序列 $(a_1, a_2, \dots, a_n)$ ，求其最大子段和

实例：给定序列 $A = (-2, 11, -4, 13, -5, -2)$

长度为1的子段和有： $-2, 11, -4, 13, -5, -2$

长度为2的子段和有： $9, 7, 9, 8, -7$

长度为3的子段和有： $5, 20, 4, 6$

长度为4的子段和有： $18, 15, 2$

长度为5的子段和有： $13, 13$

长度为6的子段和有： 11

最大子段和为 $11 - 4 + 13 = 20$

动态规划求解问题典型案例

□ 最大字段和问题 – 蛮力法

```
int MaxSum (int n, int* a, int& besti, int& bestj) {  
    int sum = MIN;  
    for (int i = 1; i <= n; i++)  
        for (int j = i; j <= n; j++){  
            int thissum = 0;  
            for (k = i; k <= j; k++) thissum += a[k];  
            if (thissum > sum) {  
                sum = thissum;  
                besti = i;  
                bestj = j;  
            }  
        }  
    return sum;  
}
```

时间复杂度 $O(n^3)$

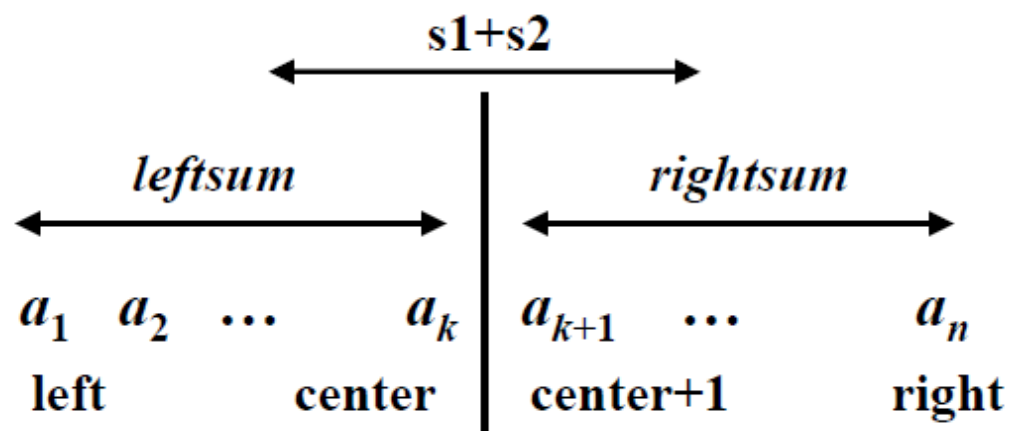
```
int MaxSum (int n, int* a, int& besti, int& bestj) {  
    int sum = MIN;  
    for (int i = 1; i <= n; i++) {  
        int thissum = 0;  
        for (int j = i; j <= n; j++){  
            thissum += a[j];  
            if (thissum > sum) {  
                sum = thissum;  
                besti = i;  
                bestj = j;  
            }  
        }  
    }  
    return sum;  
}
```

时间复杂度 $O(n^2)$

动态规划求解问题典型案例

□ 最大字段和问题 – 分治法

将序列分为左右两半，中间分点为center;
递归计算左段最大子段和leftsum;
递归计算右段最大子段和rightsum;
 $a_{\text{center}} \rightarrow a_1$ 的最大和s1, $a_{\text{center}} \rightarrow a_n$ 的最大和s2;
问题解为 $\max\{\text{leftsum}, \text{rightsum}, s1+s2\}$ 。



动态规划求解问题典型案例

□ 最大字段和问题 – 分治法

$$T(n) = \begin{cases} O(1) & n \leq c \\ 2T(n/2) + O(n) & n > c \end{cases}$$

$O(n \log n)$

```
int MaxSum (int n, int* a) {  
    return MaxSubSum (a, 1, n);  
}
```

```
int MaxSubSum (int* a, int left, int right) {  
    int sum = 0;  
    if (left == right) sum = a[left] > 0 ? a[left]:0;  
    else {  
        int center = (left + right)/2;  
        int leftsum = MaxSubSum (a, left, center);  
        int rightsum = MaxSubSum (a, center + 1, right);  
        int s1 = 0; int lefts = 0;  
        for (int i = center; i >= left; i--) {  
            lefts += a[i];  
            if (lefts > s1) s1 = lefts;  
        }  
        int s2 = 0; int rights = 0;  
        for (int i = center + 1; i <= right; i++) {  
            rights += a[i];  
            if (rights > s2) s2 = rights;  
        }  
        sum = s1 + s2;  
        if (sum < leftsum) sum = leftsum;  
        if (sum < rightsum) sum = rightsum;  
    }  
    return sum;  
}
```

动态规划求解问题典型案例

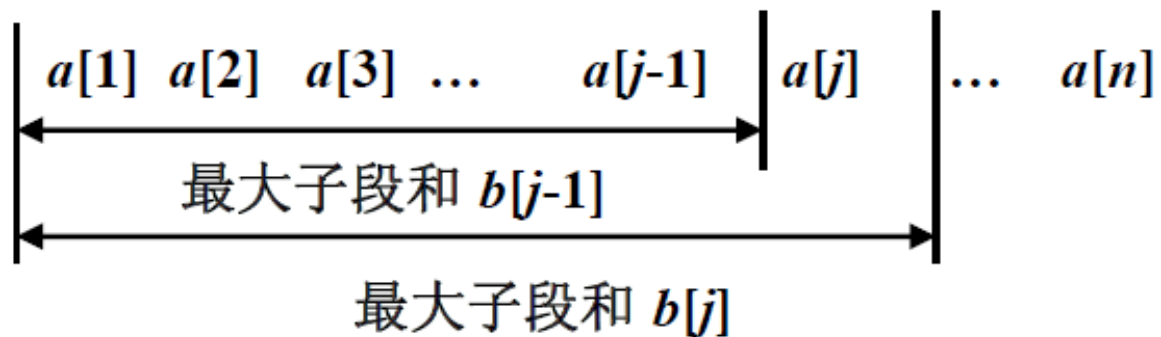
□ 最大字段和问题 – 动态规划

设 $b[j]$ 表示最后一项为 $a[j]$ 的序列构成的最大的子段和最优解 $b[1], b[2], \dots, b[n]$ 中的最大值，即

$$b[j] = \max_{1 \leq i \leq j} \left\{ \sum_{k=i}^j a[k] \right\}, 1 \leq j \leq n$$

可建立 $b[j]$ 的递推方程

$$b[j] = \max \{ b[j-1] + a[j], a[j] \}$$



动态规划求解问题典型案例

□ 最大字段和问题 – 动态规划

```
int MaxSum (int n, int* a) {  
    int sum = MIN, b = 0;  
    for (i = 1; i <= n; i++) {  
        if (b > 0) b += a[i];  
        else b = a[i];  
        if (b > sum) sum = b;  
    }  
    return sum;  
}
```

时间复杂度 $O(n)$

实例：给定序列 $A = (2, -5, 8, 11, -3, 4, 6)$

$i=1$: $b=a[i]=2$, $sum=b=2$;
 $i=2$: $b=2+a[2]=-3$, $sum=2$;
 $i=3$: $b=a[3]=8$, $sum=8$;
 $i=4$: $b=8+a[4]=19$, $sum=19$;
 $i=5$: $b=19+a[5]=16$, $sum=19$;
 $i=6$: $b=16+a[6]=20$, $sum=20$;
 $i=7$: $b=20+a[7]=26$, $sum=26$.

参考代码

动态规划求解问题典型案例

□ 0-1背包问题

给定 n 种物品和一背包。物品 i 的重量是 w_i ，其价值为 v_i ，背包的容量为 c 。

问题：应如何选择装入背包中的物品，使得装入背包中物品的总价值最大。

说明：在选择装入背包的物品时，对每种物品 i 只有两个选择，装入背包或不装入背包，也不能将物品装入背包多次。

给定 $c>0$ ， $w_i>0$ ， $v_i>0$ ， $1\leq i\leq n$ ，要求找出一个 n 元0-1向量 $(x_1, x_2, \dots, x_n)$ ， $x_i \in \{0,1\}$ ，使得

$$\sum_{i=1}^n w_i x_i \leq c \quad \text{且} \quad \sum_{i=1}^n v_i x_i \quad \text{达到最大。}$$

$$\max \sum_{k=1}^n v_k x_k \quad \begin{cases} \sum_{k=1}^n w_k x_k \leq c \\ x_k \in \{0,1\}, 1 \leq k \leq n \end{cases}$$

最优子结构性性质分析：第 i 个物品装与不装的选择情况

动态规划求解问题典型案例

□ 0-1背包问题－递推方程建立

设子问题最优值为 $m(i, j)$ ，即 $m(i, j)$ 是背包容量为 j ，可选择物品为 $1, 2, \dots, i$ 时0-1背包问题的最优值。由0-1背包问题的最优子结构性质，可以建立计算 $m(i, j)$ 的递归式如下：

$$m(i, j) = \begin{cases} \max\{m(i-1, j), m(i-1, j - w_i) + v_i\} & j \geq w_i \\ m(i-1, j) & 0 \leq j < w_i \end{cases}$$

程序设计时要防止越界！

$$\begin{cases} m(0, j) = 0 \\ m(i, 0) = 0 \end{cases}$$

动态规划求解问题典型案例

□ 0-1背包问题 – 自底向上求解最优值 | 构造最优解

时间复杂度:

Knapsack: $O(nc)$

Traceback: $O(n)$

伪多项式时间，当 $c > 2^n$ 时，算法Knapsack需要 $\Omega(n2^n)$

参考代码

动态规划求解问题典型案例

$n=5, c=10,$
 $w=\{2, 2, 6, 5, 4\}$
 $v=\{6, 3, 5, 4, 6\}$

例

0-1背包问题—算例分析

$k \backslash c$		1	2	3	4	5	6	7	8	9	10
1	$F_k(y)$	0	0	0	0	0	0	0	0	0	0
2	0	0	6	6	6	6	6	6	6	6	6
3	0	0	6	6	9	9	9	9	9	9	9
4	0	0	6	6	9	9	9	10	11	14	14
5	0	0	6	6	9	9	12	12	15	15	15

Diagram illustrating the dynamic programming table for the 0-1 knapsack problem. The table shows the maximum value $F_k(y)$ for each item k (rows) and capacity c (columns). The values are calculated based on the previous row and the current item's weight and value. Arrows indicate the transitions between cells, showing the addition of the current item's value to the previous state.

Transitions shown:

- $F_1(2) = F_0(0) + V[1]$
- $F_2(3) = F_1(1) + V[2]$
- $F_2(4) = F_1(2) + V[2]$
- $F_3(5) = F_2(2) + V[3]$
- $F_4(6) = F_3(2) + V[4]$
- $F_4(7) = F_3(3) + V[4]$
- $F_5(7) = F_4(3) + V[5]$

动态规划求解问题典型案例

□ 0-1背包问题 – 不同的递推关系

设子问题最优值为 $m(i, j)$ ，即 $m(i, j)$ 是背包容量为 j ，可选择物品为 $i, i+1, \dots, n$ 时0-1背包问题的最优值。由0-1背包问题的最优子结构性质，可以建立计算 $m(i, j)$ 的递归式如下：

$$m(i, j) = \begin{cases} \max \{m(i+1, j), m(i+1, j - w_i) + v_i\} & j \geq w_i \\ m(i+1, j) & 0 \leq j < w_i \end{cases}$$

$$m(n, j) = \begin{cases} v_n & j \geq w_n \\ 0 & 0 \leq j < w_n \end{cases}$$

动态规划求解问题典型案例

□ 0-1背包问题 – 不同递推关系建立

$n=5, c=10,$
 $w=\{2, 2, 6, 5, 4\}$
 $v=\{6, 3, 5, 4, 6\}$

例

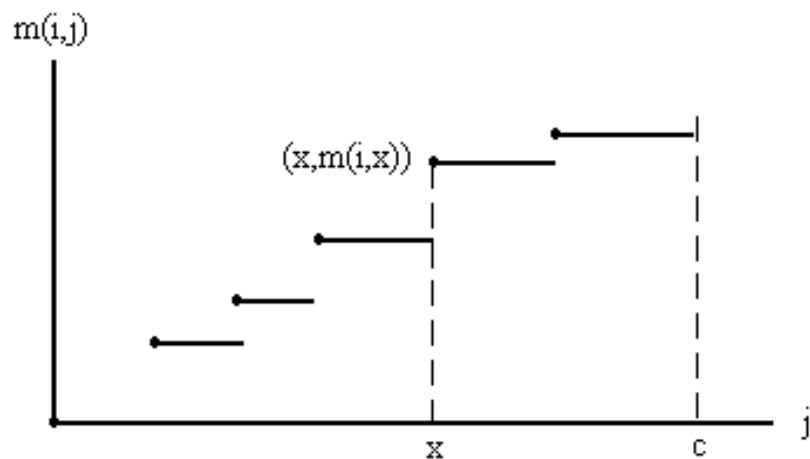
$M[i][j]$	1	2	3	4	5	6	7	8	9	10
0	0	6	6	9	9	12	12	15	15	15
0	0	3	3	6	6	9	9	9	10	11
0	0	0	0	6	6	6	6	6	10	11
0	0	0	0	6	6	6	6	6	10	10
0	0	0	0	6	6	6	6	6	6	6

Diagram illustrating the dynamic programming table for the 0-1 knapsack problem. The table shows the maximum value $M[i][j]$ for each item i (rows 1 to 5) and capacity j (columns 1 to 10). The items have weights $w = \{2, 2, 6, 5, 4\}$ and values $v = \{6, 3, 5, 4, 6\}$. The table is filled with values, and some cells are highlighted in orange to show the optimal solution path. Arrows indicate the recurrence relation: $M[i][j] = \max(M[i-1][j], M[i-1][j-w_i] + v_i)$. The optimal solution is found at $M[5][10] = 15$.

动态规划求解问题典型案例

□ 0-1背包问题 – 改进与优化

根据 $m(i,j)$ 的递归式，一般情况下，对每一个确定的 $i(1 \leq i \leq n)$ ，函数 $m(i,j)$ 是关于变量 j 的阶梯状单调不减函数。跳跃点是这一类函数的描述特征。函数 $m(i,j)$ 由其全部跳跃点唯一确定。如图所示。



对每一个确定的 $i(1 \leq i \leq n)$ ，可用一个表 $p[i]$ 存储函数 $m(i, j)$ 的全部跳跃点。表 $p[i]$ 可依计算 $m(i, j)$ 的递归式递归地由表 $p[i+1]$ 计算，初始时 $p[n+1] = \{(0, 0)\}$ 。

动态规划求解问题典型案例

□ 0-N背包问题

给定n种物品和一背包。物品i的重量是 w_i ，其价值为 v_i ，背包的容量为c。

问题：应如何选择装入背包中的物品，使得装入背包中物品的总价值最大。

说明：同一物品可以装入背包多次。

目标函数

$$\max \sum_{i=1}^n v_i x_i$$

约束条件

$$\begin{cases} \sum_{i=1}^n w_i x_i \leq c \\ x_i \in N \end{cases}$$

$$m(i,j) = \begin{cases} \max\{m(i-1,j), m(i,j-w_i) + v_i\} & j \geq w_i \\ m(i-1,j) & 0 \leq j < w_i \end{cases}$$

$$\begin{cases} m(0,j) = 0 \\ m(i,0) = 0 \end{cases}$$

参考代码

动态规划求解问题典型案例

□ 0-N背包问题 – 算例分析

$n=4, c=10$

$w=\{2, 3, 4, 7\}$

$v=\{1, 3, 5, 9\}$

例

$F_k(y)$

$k \backslash y$	1	2	3	4	5	6	7	8	8	10
1	0	1	1	2	2	3	3	4	4	5
2	0	1	3	3	4	6	6	7	9	9
3	0	1	3	5	5	6	8	10	10	11
4	0	1	3	5	5	6	9	10	10	12

Diagram illustrating the dynamic programming table for the 0-N knapsack problem. The table shows the maximum value $F_k(y)$ for each item k (rows) and capacity y (columns). The values are calculated based on the recurrence relation $F_k(y) = \max\{F_{k-1}(y), F_{k-1}(y-w_k) + v_k\}$. The final optimal value is 12, achieved by selecting items 1, 2, and 3.

Annotations on the table:

- For $k=1$, $F_1(y) = y$ for $y \leq 4$.
- For $k=2$, $F_2(y) = F_1(y-2) + v_2 = y - 1$ for $y \geq 2$.
- For $k=3$, $F_3(y) = F_2(y-3) + v_3 = y$ for $y \geq 3$.
- For $k=4$, $F_4(y) = F_3(y-4) + v_4 = y + 1$ for $y \geq 4$.

动态规划求解问题典型案例

□ 0-x背包问题

给定n种物品和一背包。物品i的重量是 w_i ，其价值为 v_i ，背包的容量为c。

问题：应如何选择装入背包中的物品，使得装入背包中物品的总价值最大。

说明：同一物品数量分别为 $x_1, x_2, \dots, x_n$ 。

目标函数

$$\max \sum_{i=1}^n v_i x_i$$

约束条件

$$\begin{cases} \sum_{i=1}^n w_i x_i \leq c \\ x_i \in (0, x_i) \end{cases}$$

$$m(i, j) = \begin{cases} \max_{0 \leq x \leq \min(x_i, j/w_i)} \{m(i-1, j-w_i x) + v_i x\} & j \geq w_i \\ m(i-1, j) & 0 \leq j < w_i \end{cases}$$

$$\begin{cases} m(0, j) = 0 \\ m(i, 0) = 0 \end{cases}$$

参考代码

动态规划求解问题典型案例

□ 投资问题

问题：m元钱，n个投资项目，函数 $f_i(x)$ 表示将x元钱投入第i个项目所产生的效益， $i=1,2,\dots,n$ 。

问题：如何分配这m元钱，使得投资的总效益最高？

目标函数： $\max \{f_1(x_1) + f_2(x_2) + \dots + f_n(x_n)\}$

约束条件： $x_1 + x_2 + \dots + x_n = m, x_i \in N$

动态规划求解问题典型案例

□ 投资问题－实例分析

有5万元钱，4个项目，效益函数（以万元为单位）如表所示。

x	$f_1(x)$	$f_2(x)$	$f_3(x)$	$f_4(x)$
0	0	0	0	0
1	11	0	2	20
2	12	5	10	21
3	13	10	30	22
4	14	15	32	23
5	15	20	40	24

动态规划求解问题典型案例

□ 投资问题－最优子结构性性质分析－递推方程建立

设 $F(k, x)$ 表示 x 万元投给前 k 个项目的最大收益，其中 $k=1,2,\dots,n$ ， $x=1,2,\dots,m$ 。可以得到投资收益函数递归关系：

$$F(k, x) = \begin{cases} f_1(x) & k = 1 \\ \max_{0 \leq x_k \leq x} \{f_k(x_k) + F(k-1, x-x_k)\} & k > 1 \end{cases}$$

参考代码

动态规划求解问题典型案例

例

□ 投资问题－算例分析

X	F(1,x) $x_1(x)$	F(2,x) $x_2(x)$	F(3,x) $x_3(x)$	F(4,x) $x_4(x)$
1	11 1	11 0	11 0	20 1
2	12 2	12 0	13 1	31 1
3	13 3	16 2	30 3	33 1
4	14 4	21 3	41 3	50 1
5	15 5	26 4	43 4	61 1

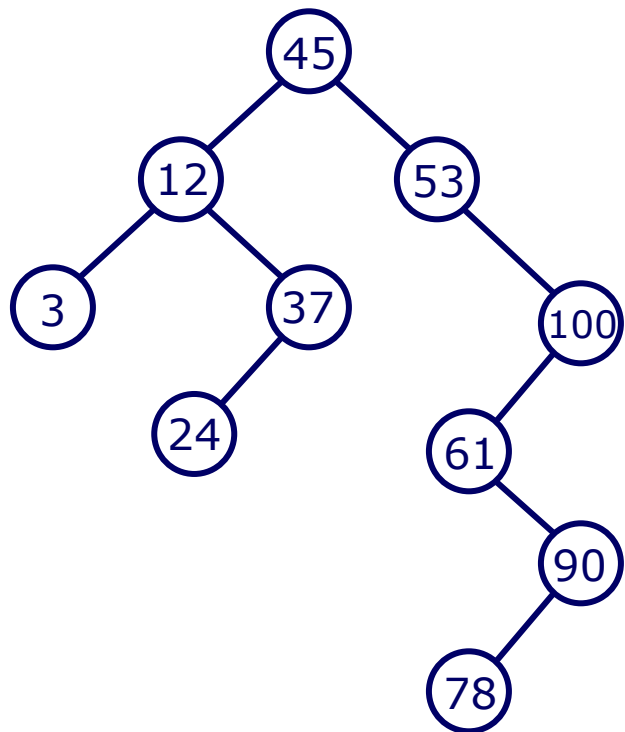
最优解: $x_1=1, x_2=0, x_3=3, x_4=1$;
 $F(4,5)=61$.

动态规划求解问题典型案例

□ 最优二叉搜索树问题

二叉排序树:

- (1)若它的左子树不空，则左子树上所有节点的值均小于它的根节点的值;
- (2)若它的右子树不空，则右子树上所有节点的值均大于它的根节点的值;
- (3)它的左、右子树也分别为二叉排序树。



随机的情况下，二叉查找树的平均查找长度和 $\log n$ 是等数量级的。

动态规划求解问题典型案例

□ 最优二叉搜索树问题

二叉搜索树的叶结点是形如 (x_i, x_{i+1}) 的开区间。

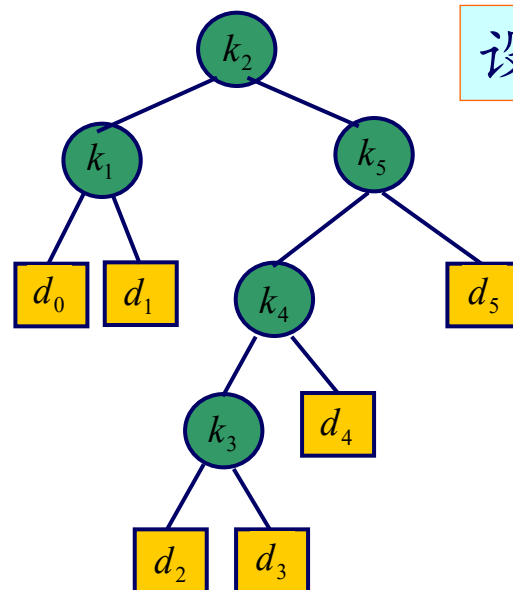
在二叉搜索树中搜索一个元素 x ，返回结果有两种情形：

- (1) 在树的内结点找到 $x=x_i$;
- (2) 在树的叶结点中确定 $x \in (x_i, x_{i+1})$ 。

$$\sum_{i=1}^n p_i + \sum_{i=0}^n q_i = 1$$

$E(\text{search cost in } T)$

$$= \sum_{i=1}^n (\text{depth}_T(k_i) + 1) \cdot p_i + \sum_{i=0}^n \text{depth}_T(d_i) \cdot q_i$$



设树的根节点深度为0

动态规划求解问题典型案例

□ 最优二叉搜索树问题

设数据集 $S=\{a, b, c, d, e\}$ ，存储 S 的二叉搜索树如图所示。
设被查找的 x 值对应树中各结点的分布概率为
 $P=(4, 10, 2, 30, 2, 10, 5, 20, 6, 10, 1)/100$

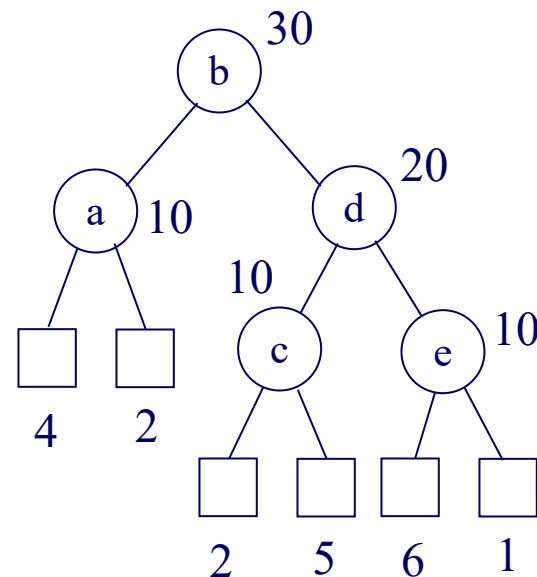
平均查找长度：

$$\begin{aligned} L &= 0.3 \times 1 + (0.1 + 0.2) \times 2 + (0.1 + 0.1) \times 3 + (0.04 + 0.02) \times 2 \\ &\quad + (0.02 + 0.05 + 0.06 + 0.01) \times 3 \\ &= 2.04 \end{aligned}$$

这棵树构造的是否最好？

给定数据集 S 与存取概率分布 P ，求一颗平均查找次数（平均查找长度）最小的二叉搜索树，即最优二叉搜索树。

蛮力法？复杂度与矩阵链相乘问题相同



动态规划求解问题典型案例

□ 最优二叉搜索树问题 – 最优子结构性质的分析

设 $s[i][j] = \langle x_i, x_{i+1}, \dots, x_j \rangle$ 是 s 以 i 和 j 作为边界的子数据集，
 $p[i][j] = \langle a_{i-1}, b_i, a_i, b_{i+1}, \dots, b_j, a_j \rangle$ 是对应 $s[i][j]$ 的存取概率分布。

例： $s = \langle a, \underline{b}, \underline{c}, \underline{d}, e \rangle$, $p = \langle 4, \textcolor{red}{10}, 2, \textcolor{red}{30}, 2, \textcolor{red}{10}, 5, \textcolor{red}{20}, 6, \textcolor{red}{10}, 1 \rangle / 100$
 $s[2][4] = \langle b, c, d \rangle$, $p[2][4] = \langle 2, \textcolor{red}{30}, 2, \textcolor{red}{10}, 5, \textcolor{red}{20}, 6 \rangle / 100$

设以 x_k 为根，可将原问题划分为两个子问题：

$s[i][k-1]$, $p[i][k-1]$

$s[k+1][j]$, $p[k+1][j]$

例如以 b 为根，可将 s , p 划分为

$s[1][1] = \langle a \rangle$, $p[1][1] = \langle 4, \textcolor{red}{10}, 2 \rangle / 100$

$s[3][5] = \langle c, d, e \rangle$, $p[3][5] = \langle 2, \textcolor{red}{10}, 5, \textcolor{red}{20}, 6, \textcolor{red}{10}, 1 \rangle / 100$

动态规划求解问题典型案例

□ 最优二叉搜索树问题 – 递推关系建立

设 $m[i][j]$ 是相对于输入 $s[i][j]$ 和 $p[i][j]$ 的最优二叉搜索树的平均比较次数, 令 $p[i][j]$ 中所有概率 (包括数据元素与空隙) 之和为 $w[i,j]$, 则

$$w[i,j] = \sum_{p=i-1}^j a_p + \sum_{q=i}^j b_q$$

$$m[i,j] = \min_{i \leq k \leq j} \{m[i,k-1] + m[k+1,j] + w[i][j]\}, 1 \leq i \leq j \leq n$$

$$\begin{aligned} m[i,i-1] &= 0, \quad i = 1, 2, \dots, n & m[i,j]_k &= (m[i,k-1] + w[i,k-1]) + (m[k+1,j] + w[k+1,j]) + 1 \times b_k \\ & & &= (m[i,k-1] + m[k+1,j]) + (w[i,k-1] + b_k + w[k+1,j]) \end{aligned}$$

与矩阵链乘积问题、凸多边形最优三角剖分问题同构

$$= (m[i,k-1] + m[k+1,j]) + \left(\sum_{p=i-1}^{k-1} a_p + \sum_{q=i}^{k-1} b_q \right) + b_k + \left(\sum_{p=k}^j a_p + \sum_{q=k+1}^j b_q \right)$$

$$= (m[i,k-1] + m[k+1,j]) + \sum_{p=i-1}^j a_p + \sum_{q=i}^j b_q$$

$$= m[i,k-1] + m[k+1,j] + w[i,j]$$

动态规划求解问题典型案例

□ 最优二叉搜索树问题 – 程序设计

```
void OptimalBinarySearchTree (int a, int b, int n, int** m, int** s, int** w){
    for (int i = 0; i <= n; i++) {w[i+1][i] = a[i]; m[i+1][i] = 0;}
    for (int r = 0; r < n; r++)
        for (int i = 1; i <= n-r; i++) {
            int j = i + r;
            w[i][j] = w[i][j-1] + a[j] + b[j];
            m[i][j] = m[i+1][j];
            s[i][j] = i;
            for (int k = i+1; k <= j; k++){
                int t = m[i][k-1] + m[k+1][j];
                if (t < m[i][j]) {m[i][j] = t; s[i][j] = k;}
            }
            m[i][j] += w[i][j];
        }
}
```

时间复杂度 $O(n^3)$
空间复杂的 $O(n^2)$

$s[i][j]$ 保存最优子树 $T(i, j)$ 的根节点中元素，由 $s[i][j]$ 可在 $O(n)$ 时间内构造所求的最优二叉搜索树。

动态规划求解问题典型案例

$s=\langle A, B, C, D, E \rangle,$

$p=\langle 0.04, 0.1, 0.02, 0.3, 0.02, 0.1, 0.05, 0.2, 0.06, 0.1, 0.01 \rangle$

□ 最优二叉搜索树问题 – 算例分析

例

$r=0: m[1][1]=0.16, m[2][2]=0.34, m[3][3]=0.17, m[4][4]=0.31, m[5][5]=0.17;$

$r=1:$

$m[1][2]=\min\{m[2][2], m[1][1]\}+0.48=0.64, s[1][2]=2;$

$m[2][3]=\min\{m[3][3], m[2][2]\}+0.49=0.66, s[2][3]=2;$

$m[3][4]=\min\{m[4][4], m[3][3]\}+0.43=0.60, s[3][4]=4;$

$m[4][5]=\min\{m[5][5], m[4][4]\}+0.42=0.59, s[4][5]=4;$

$r=2:$

$m[1][3]=\min\{m[2][3], m[1][1]+m[3][3], m[1][2]\}+0.63=0.96, s[1][3]=2;$

$m[2][4]=\min\{m[3][4], m[2][2]+m[4][4], m[2][3]\}+0.75=1.35, s[2][4]=2;$

$m[3][5]=\min\{m[4][5], m[3][3]+m[5][5], m[3][4]\}+0.54=0.88, s[2][4]=4;$

$r=3:$

$m[1][4]=\min\{m[2][4], m[1][1]+m[3][4], m[1][2]+m[4][4], m[1][3]\}+0.89=1.65, s[1][4]=2;$

$m[2][5]=\min\{m[3][5], m[2][2]+m[4][5], m[2][3]+m[5][5], m[2][4]\}+0.86=1.69, s[1][4]=4;$

$r=4:$

$m[1][5]=\min\{m[2][5], m[1][1]+m[3][5], m[1][2]+m[4][5], m[1][3]+m[5][5], m[1][4]\}+1=2.04, s[1][5]=2.$

动态规划求解问题典型案例

□ 流水作业调度问题

n 个作业 $\{1, 2, \dots, n\}$ 要在由2台机器 M_1 和 M_2 组成的流水线上完成加工。每个作业加工的顺序都是先在 M_1 上加工，然后在 M_2 上加工。 M_1 和 M_2 加工作业 i 所需的时间分别为 a_i 和 b_i 。

问题：流水作业调度问题要求确定这 n 个作业的最优加工顺序，使得从第一个作业在机器 M_1 上开始加工，到最后一个作业在机器 M_2 上加工完成所需的时间最少。

○ (a_1, b_1)

○ (a_2, b_2)

...

○ (a_n, b_n)



动态规划求解问题典型案例

□ 流水作业调度问题

(1) 直观上, 一个最优调度应使机器 M_1 没有空闲时间, 且机器 M_2 的空闲时间最少。在一般情况下, 机器 M_2 上会有机器空闲和作业积压2种情况。

(2) 设全部作业的集合为 $N=\{1, 2, \dots, n\}$ 。 $S \subseteq N$ 是 N 的作业子集。在一般情况下, 机器 M_1 开始加工 S 中作业时, 机器 M_2 还在加工其它作业, 要等时间 t 后才可利用。将这种情况下完成 S 中作业所需的最短时间记为 $T(S, t)$ 。流水作业调度问题的最优值为 $T(N, 0)$ 。

动态规划求解问题典型案例

□ 流水作业调度问题

设 π 是所给 n 个流水作业的一个最优调度，它所需的加工时间为 $a_{\pi(1)}+T'$ 。其中 T' 是在机器 M_2 的等待时间为 $b_{\pi(1)}$ 时，安排作业 $\pi(2), \dots, \pi(n)$ 所需的时间。记 $S=N-\{\pi(1)\}$ ，则有 $T'=T(S, b_{\pi(1)})$ 。

证明：事实上，由 T 的定义知 $T' \geq T(S, b_{\pi(1)})$ 。若 $T' > T(S, b_{\pi(1)})$ ，设 π' 是作业集 S 在机器 M_2 的等待时间为 $b_{\pi(1)}$ 情况下的一个最优调度。则 $\pi(1), \pi'(2), \dots, \pi'(n)$ 是 N 的一个调度，且该调度所需的时间为 $a_{\pi(1)}+T(S, b_{\pi(1)}) < a_{\pi(1)}+T'$ 。这与 π 是 N 的最优调度矛盾。故 $T' \leq T(S, b_{\pi(1)})$ 。从而 $T'=T(S, b_{\pi(1)})$ 。这就证明了流水作业调度问题具有最优子结构的性质。

动态规划求解问题典型案例

□ 流水作业调度问题－递推关系建立

由流水作业调度问题的最优子结构性质分析可知：

$$T(N,0) = \min_{1 \leq i \leq n} \{a_i + T(N - \{i\}, b_i)\}$$

推广到一般情况：

$$T(S,t) = \min_{i \in S} \{a_i + T(S - \{i\}, b_i + \max\{t - a_i, 0\})\}$$

动态规划求解问题典型案例

□ 流水作业调度问题—Johnson不等式

设 π 是作业集 S 在机器 M_2 的等待时间为 t 时的任一最优调度。若 $\pi(1)=i, \pi(2)=j$ 。则由动态规划递归式可得:

$$T(S,t)=a_i+T(S-\{i\},b_i+\max\{t-a_i,0\})=a_i+a_j+T(S-\{i,j\},t_{ij})$$

其中,

$$\begin{aligned}t_{ij} &= b_j + \max\{b_i + \max\{t - a_i, 0\} - a_j, 0\} \\&= b_j + b_i - a_j + \max\{\max\{t - a_i, 0\}, a_j - b_i\} \\&= b_j + b_i - a_j + \max\{t - a_i, a_j - b_i, 0\} \\&= b_j + b_i - a_j - a_i + \max\{t, a_i + a_j - b_i, a_i\}\end{aligned}$$

如果作业 i 和 j 满足 $\min\{b_i, a_j\} \geq \min\{b_j, a_i\}$ ，则称作业 i 和 j 满足**Johnson不等式**。

动态规划求解问题典型案例

□ 流水作业调度问题—Johnson不等式

交换作业*i*和作业*j*的加工顺序，得到作业集*S*的另一调度，它所需的加工时间为
 $T'(S,t)=a_i+a_j+T(S-\{i,j\},t_{ji})$ ，其中

$$t_{ji} = b_j + b_i - a_j - a_i + \max\{t, a_i + a_j - b_j, a_j\}$$

当作业*i*和*j*满足Johnson不等式时，有

$$\max\{-b_i, -a_j\} \leq \max\{-b_j, -a_i\}$$

$$a_i + a_j + \max\{-b_i, -a_j\} \leq a_i + a_j + \max\{-b_j, -a_i\}$$

$$\max\{a_i + a_j - b_i, a_i\} \leq \max\{a_i + a_j - b_j, a_j\}$$

$$\max\{t, a_i + a_j - b_i, a_i\} \leq \max\{t, a_i + a_j - b_j, a_j\}$$

从而， $T_{ij} \leq T_{ji}$ ， $T(S,t) \leq T'(S,t)$

动态规划求解问题典型案例

□ 流水作业调度问题—Johnson不等式

当作业*i*和作业*j*不满足Johnson不等式时，交换它们的加工顺序后，不增加加工时间。对于流水作业调度问题，必存在最优调度 π ，使得作业 $\pi(i)$ 和 $\pi(i+1)$ 满足Johnson不等式。进一步还可以证明，调度满足Johnson法则当且仅当对任意*i*<*j*有

$$\min \{b_{\pi(i)}, a_{\pi(j)}\} \geq \min \{b_{\pi(j)}, a_{\pi(i)}\}$$

所有满足Johnson法则的调度均为最优调度。

动态规划求解问题典型案例

□ 流水作业调度问题—Johnson算法

- (1) 令 $N_1 = \{i \mid a_i < b_i\}, N_2 = \{i \mid a_i \geq b_i\}$
- (2) 将 N_1 中作业依 a_i 的非减序排序；将 N_2 中作业依 b_i 的非增序排序；
- (3) N_1 中作业接 N_2 中作业构成满足Johnson法则的最优调度。

算法复杂度分析:

算法的主要计算时间花在对作业集的排序。因此，在最坏情况下算法所需的计算时间为 $O(n \log n)$ 。所需的空间为 $O(n)$ 。



动态规划求解问题典型案例

□ ...

目录

CONTENTS



1

动态规划基本思想

2

动态规划适用条件

3

动态规划求解问题基本步骤

4

动态规划求解问题典型案例

5

开放性讨论

6

总结

开放性讨论

- 动态规划算法的核心思想是什么？
- 哪些问题适合采用动态规划算法求解？
- 动态规划算法求解问题的关键步骤是什么？
- 动态规划算法程序实现与分治法的区别是什么？
- 动态规划算法程序实现的特点是什么？

关于本讲内容



祝各位学习愉快!

感谢观看！

讲解人：陆伟